

CM0081 Formal Languages and Automata

§ 3.2 Finite Automata and Regular Expressions

Andrés Sicard-Ramírez

Universidad EAFIT

Semester 2024-1

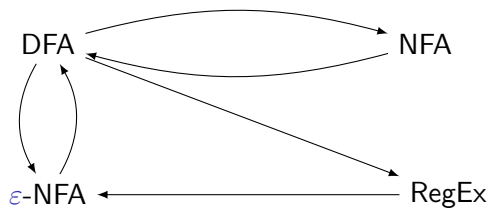
Preliminaries

Conventions

- The number and page numbers assigned to chapters, examples, exercises, figures, quotes, sections and theorems on these slides correspond to the numbers assigned in the textbook (Hopcroft, Motwani and Ullman 2007).
- The natural numbers include the zero, that is, $\mathbb{N} = \{0, 1, 2, \dots\}$.
- The power set of a set A , that is, the set of its subsets, is denoted by $\mathcal{P}A$.

Introduction

Equivalences



From Finite Automata to Regular Expressions

Theorem 3.4

If $L = L(D)$ for some DFA D , then there is a regular expression R such that $L = L(R)$.

From Finite Automata to Regular Expressions

Proof (by induction on the number of states of the automaton)

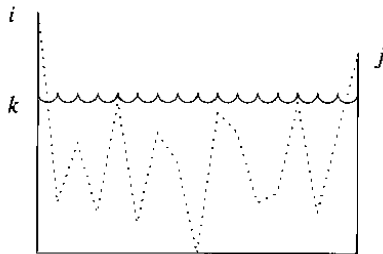
- Let the states of D be $\{1, 2, \dots, n\}$ with 1 the start state.

[†]Figure from Hopcroft, Motwani and Ullman (2007, Fig. 3.2).

From Finite Automata to Regular Expressions

Proof (by induction on the number of states of the automaton)

- Let the states of D be $\{1, 2, \dots, n\}$ with 1 the start state.
- R_{ij}^k : Regular expression describing the set of labels of all paths in D from state i to state j such that the path has no intermediate node whose number is greater than k .[†]



[†]Figure from Hopcroft, Motwani and Ullman (2007, Fig. 3.2).

From Finite Automata to Regular Expressions

Proof (continuation)

Basis step: Proof for $k = 0$ (i.e. no intermediate states) and $i \neq j$

From Finite Automata to Regular Expressions

Proof (continuation)

Basis step: Proof for $k = 0$ (i.e. no intermediate states) and $i \neq j$

- No transition from state i to state j



$$R_{ij}^0 = \emptyset$$

From Finite Automata to Regular Expressions

Proof (continuation)

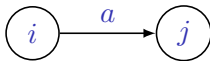
Basis step: Proof for $k = 0$ (i.e. no intermediate states) and $i \neq j$

- No transition from state i to state j



$$R_{ij}^0 = \emptyset$$

- One transition from state i to state j



$$R_{ij}^0 = a$$

From Finite Automata to Regular Expressions

Proof (continuation)

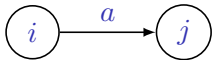
Basis step: Proof for $k = 0$ (i.e. no intermediate states) and $i \neq j$

- No transition from state i to state j



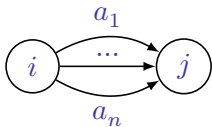
$$R_{ij}^0 = \emptyset$$

- One transition from state i to state j



$$R_{ij}^0 = a$$

- Various transitions from state i to state j



$$R_{ij}^0 = a_1 + \dots + a_n$$

From Finite Automata to Regular Expressions

Proof (continuation)

Basis step: Proof for $k = 0$ (i.e. no intermediate states) and $i = j$

From Finite Automata to Regular Expressions

Proof (continuation)

Basis step: Proof for $k = 0$ (i.e. no intermediate states) and $i = j$

- No loops



$$R_{ii}^0 = \varepsilon$$

From Finite Automata to Regular Expressions

Proof (continuation)

Basis step: Proof for $k = 0$ (i.e. no intermediate states) and $i = j$

- No loops



$$R_{ii}^0 = \varepsilon$$

- One loop



$$R_{ii}^0 = \varepsilon + a$$

From Finite Automata to Regular Expressions

Proof (continuation)

Basis step: Proof for $k = 0$ (i.e. no intermediate states) and $i = j$

- No loops



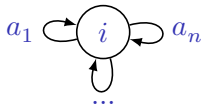
$$R_{ii}^0 = \varepsilon$$

- One loop



$$R_{ii}^0 = \varepsilon + a$$

- Various loops



$$R_{ii}^0 = \varepsilon + a_1 + \dots + a_n$$

From Finite Automata to Regular Expressions

Proof (continuation)

Basis step: Proof for $k = 0$ (i.e. no intermediate states) and $i = j$

- No loops



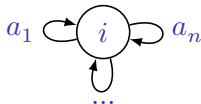
$$R_{ii}^0 = \varepsilon$$

- One loop



$$R_{ii}^0 = \varepsilon + a$$

- Various loops



$$R_{ii}^0 = \varepsilon + a_1 + \dots + a_n$$

Question

Why do not to define $R_{ii}^0 = a^*$?

From Finite Automata to Regular Expressions

Proof (continuation)

Inductive step: Proof for k

Inductive hypothesis R_{ij}^{k-1} : Path from state i to state j that goes through no state higher than $k - 1$.

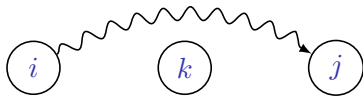
From Finite Automata to Regular Expressions

Proof (continuation)

Inductive step: Proof for k

Inductive hypothesis R_{ij}^{k-1} : Path from state i to state j that goes through no state higher than $k-1$.

- The path does not go through state k at all



$$R_{ij}^k = R_{ij}^{k-1}$$

From Finite Automata to Regular Expressions

Proof (continuation)

Inductive step: Proof for k

Inductive hypothesis R_{ij}^{k-1} : Path from state i to state j that goes through no state higher than $k - 1$.

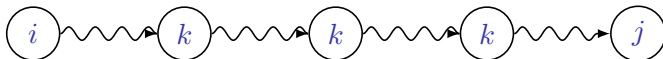
From Finite Automata to Regular Expressions

Proof (continuation)

Inductive step: Proof for k

Inductive hypothesis R_{ij}^{k-1} : Path from state i to state j that goes through no state higher than $k-1$.

- The path goes through state k at least once



$$R_{ij}^k = R_{ik}^{k-1} (R_{kk}^{k-1})^* R_{kj}^{k-1}$$

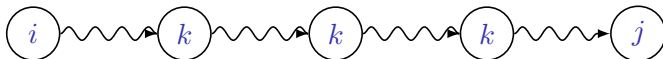
From Finite Automata to Regular Expressions

Proof (continuation)

Inductive step: Proof for k

Inductive hypothesis R_{ij}^{k-1} : Path from state i to state j that goes through no state higher than $k-1$.

- The path goes through state k at least once



$$R_{ij}^k = R_{ik}^{k-1} (R_{kk}^{k-1})^* R_{kj}^{k-1}$$

From the previous cases:

$$R_{ij}^k = R_{ij}^{k-1} + R_{ik}^{k-1} (R_{kk}^{k-1})^* R_{kj}^{k-1}.$$

From Finite Automata to Regular Expressions

Proof (continuation)

Given that the states of D are $\{1, 2, \dots, n\}$ with 1 the start state, then

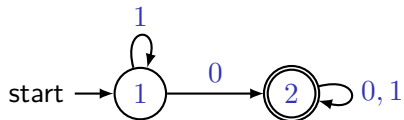
$$L(D) = L(R_{1f_1}^n + \dots + R_{1f_m}^n) \text{ with } f_i \in F.$$



From Finite Automata to Regular Expressions

Example

To convert the DFA D to a regular expression.



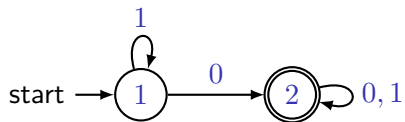
$$L(D) = \{ w \in \{0, 1\}^* \mid w \text{ has at least one } 0 \}.$$

(continued on next slide)

From Finite Automata to Regular Expressions

Example (continuation)

- R_{ij}^0



R_{ij}^0	Regex
R_{11}^0	$\varepsilon + \mathbf{1}$
R_{12}^0	$\mathbf{0}$
R_{21}^0	$\emptyset$
R_{22}^0	$\varepsilon + \mathbf{0} + \mathbf{1}$

(continued on next slide)

From Finite Automata to Regular Expressions

Example (continuation)

- $R_{ij}^1 = R_{ij}^0 + R_{i1}^0 (R_{11}^0)^* R_{1j}^0$

R_{ij}^0	Regexp
------------	--------

R_{11}^0	$\varepsilon + \mathbf{1}$
------------	----------------------------

R_{12}^0	$\mathbf{0}$
------------	--------------

R_{21}^0	$\emptyset$
------------	-------------

R_{22}^0	$\varepsilon + \mathbf{0} + \mathbf{1}$
------------	---

R_{ij}^1	Regexp
------------	--------

R_{11}^1	$(\varepsilon + \mathbf{1}) + (\varepsilon + \mathbf{1})(\varepsilon + \mathbf{1})^*(\varepsilon + \mathbf{1})$
------------	---

R_{12}^1	$\mathbf{0} + (\varepsilon + \mathbf{1})(\varepsilon + \mathbf{1})^*\mathbf{0}$
------------	---

R_{21}^1	$\emptyset + \emptyset(\varepsilon + \mathbf{1})^*(\varepsilon + \mathbf{1})$
------------	---

R_{22}^1	$\varepsilon + \mathbf{0} + \mathbf{1} + \emptyset(\varepsilon + \mathbf{1})^*\mathbf{0}$
------------	---

(continued on next slide)

From Finite Automata to Regular Expressions

Example (continuation)

- Some simplifications for regular expressions

Let M and N be regular expression variables.

$$(\varepsilon + M)^* = M^*$$

$$(\varepsilon + M)M^* = M^*$$

$$M + N^*M = N^*M$$

$$M\emptyset = \emptyset M = \emptyset$$

($\emptyset$ is the annihilator for concatenation)

$$M + \emptyset = \emptyset + M = M$$

($\emptyset$ is the identity for union)

(continued on next slide)

From Finite Automata to Regular Expressions

Example (continuation)

- $R_{ij}^1 = R_{ij}^0 + R_{i1}^0 (R_{11}^0)^* R_{1j}^0$

R_{ij}^1	Regexp	Simplified
R_{11}^1	$(\varepsilon + \mathbf{1}) + (\varepsilon + \mathbf{1})(\varepsilon + \mathbf{1})^*(\varepsilon + \mathbf{1})$	$\mathbf{1}^*$
R_{12}^1	$\mathbf{0} + (\varepsilon + \mathbf{1})(\varepsilon + \mathbf{1})^*\mathbf{0}$	$\mathbf{1}^*\mathbf{0}$
R_{21}^1	$\emptyset + \emptyset(\varepsilon + \mathbf{1})^*(\varepsilon + \mathbf{1})$	$\emptyset$
R_{22}^1	$\varepsilon + \mathbf{0} + \mathbf{1} + \emptyset(\varepsilon + \mathbf{1})^*\mathbf{0}$	$\varepsilon + \mathbf{0} + \mathbf{1}$

(continued on next slide)

From Finite Automata to Regular Expressions

Example (continuation)

- $R_{ij}^2 = R_{ij}^1 + R_{i2}^1(R_{22}^1)^*R_{2j}^1$

R_{ij}^1	Regexp	R_{ij}^2	Regexp
R_{11}^1	$\mathbf{1}^*$	R_{11}^2	$\mathbf{1}^* + \mathbf{1}^*\mathbf{0}(\varepsilon + \mathbf{0} + \mathbf{1})^*\emptyset$
R_{12}^1	$\mathbf{1}^*\mathbf{0}$	R_{12}^2	$\mathbf{1}^*\mathbf{0} + \mathbf{1}^*\mathbf{0}(\varepsilon + \mathbf{0} + \mathbf{1})^*(\varepsilon + \mathbf{0} + \mathbf{1})$
R_{21}^1	$\emptyset$	R_{21}^2	$\emptyset + (\varepsilon + \mathbf{0} + \mathbf{1})(\varepsilon + \mathbf{0} + \mathbf{1})^*\emptyset$
R_{22}^1	$\varepsilon + \mathbf{0} + \mathbf{1}$	R_{22}^2	$\varepsilon + \mathbf{0} + \mathbf{1} + (\varepsilon + \mathbf{0} + \mathbf{1})(\varepsilon + \mathbf{0} + \mathbf{1})^*(\varepsilon + \mathbf{0} + \mathbf{1})$

(continued on next slide)

From Finite Automata to Regular Expressions

Example (continuation)

- $R_{ij}^2 = R_{ij}^1 + R_{i2}^1(R_{22}^1)^*R_{2j}^1$

R_{ij}^2	Regexp	Simplified
R_{11}^2	$\mathbf{1}^* + \mathbf{1}^*\mathbf{0}(\varepsilon + \mathbf{0} + \mathbf{1})^*\emptyset$	$\mathbf{1}^*$
R_{12}^2	$\mathbf{1}^*\mathbf{0} + \mathbf{1}^*\mathbf{0}(\varepsilon + \mathbf{0} + \mathbf{1})^*(\varepsilon + \mathbf{0} + \mathbf{1})$	$\mathbf{1}^*\mathbf{0}(\mathbf{0} + \mathbf{1})^*$
R_{21}^2	$\emptyset + (\varepsilon + \mathbf{0} + \mathbf{1})(\varepsilon + \mathbf{0} + \mathbf{1})^*\emptyset$	$\emptyset$
R_{22}^2	$\varepsilon + \mathbf{0} + \mathbf{1} + (\varepsilon + \mathbf{0} + \mathbf{1})(\varepsilon + \mathbf{0} + \mathbf{1})^*(\varepsilon + \mathbf{0} + \mathbf{1})$	$(\mathbf{0} + \mathbf{1})^*$

(continued on next slide)

From Finite Automata to Regular Expressions

Example (continuation)

The regular expression equivalent to the automaton D is

$$R_{12}^2 = 1^*0(0 + 1)^*.$$

From Regular Expressions to Finite Automata

Theorem 3.7

Every language defined by a regular expression is also defined by a finite automaton.

From Regular Expressions to Finite Automata

Theorem 3.7

Every language defined by a regular expression is also defined by a finite automaton.

Proof (by induction)

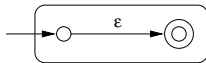
For each regular expression E we construct an ε -NFA A such that $L(E) = L(A)$ with:

- (i) exactly one accepting state,
- (ii) no arcs into the initial state and
- (iii) no arcs out of the accepting state.

From Regular Expressions to Finite Automata

Proof (continuation)

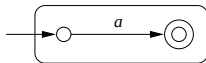
Basis step: Automata for (a) $E = \varepsilon$, (b) $E = \emptyset$ and (c) $E = a$.[†]



(a)



(b)



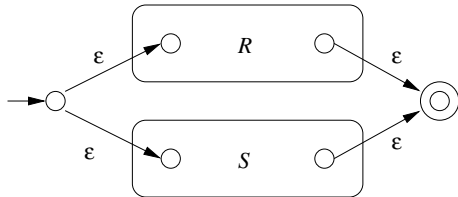
(c)

[†]Figure from Hopcroft, Motwani and Ullman (2007, Fig. 3.16).

From Regular Expressions to Finite Automata

Proof (continuation)

Inductive step: Automaton for $E = R + S$.[†]

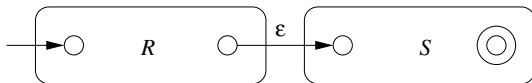


[†]Figure from Hopcroft, Motwani and Ullman (2007, Fig. 3.17).

From Regular Expressions to Finite Automata

Proof (continuation)

Inductive step: Automaton for $E = RS$.[†]

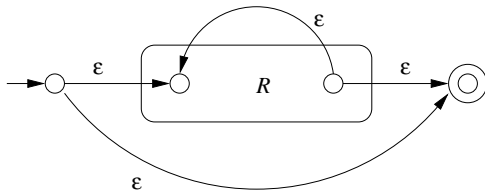


[†]Figure from Hopcroft, Motwani and Ullman (2007, Fig. 3.17).

From Regular Expressions to Finite Automata

Proof (continuation)

Inductive step: Automaton for $E = R^*.$ [†]



[†]Figure from Hopcroft, Motwani and Ullman (2007, Fig. 3.17).

From Regular Expressions to Finite Automata

Example

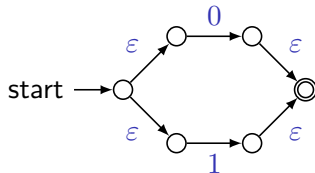
Convert the regular expression $E = (0+1)^*1(0+1)$ to an ε -NFA A such that $L(E) = L(A)$.

From Regular Expressions to Finite Automata

Example

Convert the regular expression $E = (0+1)^*1(0+1)$ to an ε -NFA A such that $L(E) = L(A)$.

1. ε -NFA for $0 + 1$:

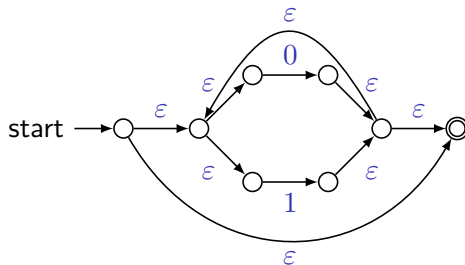


(continued on next slide)

From Regular Expressions to Finite Automata

Example (continuation)

2. ϵ -NFA for $(0 + 1)^*$:

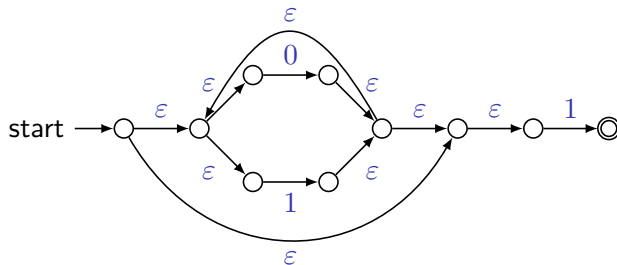


(continued on next slide)

From Regular Expressions to Finite Automata

Example (continuation)

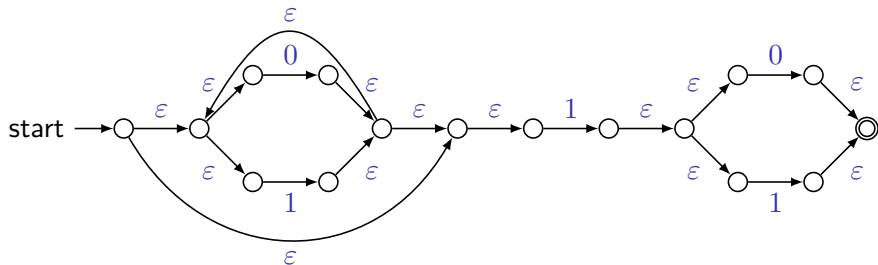
3. ϵ -NFA for $(0 + 1)^*1$:



From Regular Expressions to Finite Automata

Example (continuation)

4. ϵ -NFA for $(0 + 1)^*1(0 + 1)$:



From Regular Expressions to Finite Automata

Exercise 3.2.7

There are some simplifications to the constructions of Theorem 3.7, where we converted a regular expression to an ϵ -NFA. Here are three:

- 1) For the union operator, instead of creating new start and accepting states, merge the two start states into one state with all the transitions of both start states. Likewise, merge the two accepting states, having all transitions to either go to the merged state instead.
- 2) For the concatenation operator, merge the accepting state of the first automaton with the start state of the second.
- 3) For the closure operator, simply add ϵ -transitions from the accepting state to the start state and vice-versa.

(continued on next slide)

Exercise 3.2.7 (continuation)

Each of these simplifications, by themselves, still yield a correct construction; that is, the resulting ϵ -NFA for any regular expression accepts the language of the expression. Which subsets of changes 1), 2) and 3) may be made to the construction together, while still yielding a correct automaton for every regular expression?

References



John E. Hopcroft, Rajeev Motwani and Jefferey D. Ullman [1979] (2007). Introduction to Automata Theory, Languages, and Computation. 3rd ed. Pearson Education (cit. on pp. 2, 5, 6, 32–35).